

Biosensor-to-LLM Optimization

Complete Project Context for AI-Assisted Analysis

This document contains the full architecture, source code structure, design decisions, security model, analytics engine, and current implementation state of the biosensor-to-LLM MCP server. Upload this to any Claude conversation to gain instant project context.

Property	Value
Version	3.0.0
Architecture	Parent/Child MCP with 5-layer security pipeline
Language	Python 3.10+ (mcp, requests, orjson)
Transport	stdio (Claude Desktop)
Reference Implementation	Strava running data (13 tools, 3 tiers)
License	MIT

Contents

1. Project Overview & Framing
2. Architecture: Parent/Child MCP Pattern
3. Security Pipeline (5 Middleware Layers)
4. Three-Tier Access Model
5. Framework Layer (Domain-Agnostic)
6. Running Child Implementation
7. Server-Side Analytics Engine
8. Strava API & Token Management
9. CLI & Installation
10. Key Design Decisions & Rationale
11. Known Issues & Diagnostic Findings
12. Extensibility: Adding New Biosensor Children
13. Full Source File Index

1. Project Overview & Framing

This project is NOT a Strava coaching product. It is a **reference implementation** for a general problem: how to efficiently pipe high-frequency biosensor data (HR, GPS, pace, altitude, glucose, SpO2) into LLM context windows without blowing token budgets or leaking sensitive biometric data unnecessarily.

Strava running data is the proving ground, but the architecture (three-tier access, server-side computation, downsampling, precision reduction, consent gates, cost gates) is meant to generalize to any biosensor-to-LLM pipeline: CGM, sleep staging, ECG, lab results.

The Core Problem

Biosensor devices produce per-second data streams. A 15-mile run generates ~8,600 data points across 8 stream types, serializing to ~200,000 tokens. Sending that to an LLM for every question is wasteful (200K tokens vs 800 needed), expensive (~\$60/month vs \$0.02 with server-side computation), and a privacy risk (raw HR + GPS = re-identification).

Token Savings Achieved

Scenario	Raw Tokens	Optimized	Reduction
Single run analysis	200,000	800	99.6%
Compare 3 runs	600,000	1,500	99.75%
Monthly trend (30 runs)	2,000,000	600	99.97%

2. Architecture: Parent/Child MCP Pattern

A **parent router MCP** handles security, consent, and cost gating. **Child MCPs** handle domain-specific data and analytics. The parent never touches raw sensor data — it only routes requests through the security pipeline.

```
Claude Desktop <--> Parent Router (validate -> circuit break -> consent -> cost -> audit)
|
Child MCP (domain-specific execution)
e.g. RunningChild, CGMChild, SleepChild
```

File Structure

```
src/strava_coach/
__init__.py # Package metadata (v3.0.0)
__main__.py # CLI: serve, setup, status, uninstall
wizard.py # OAuth setup wizard (browser-based)
framework/ # Domain-agnostic framework
__init__.py # Public API exports
interfaces.py # ChildMCP base class, ToolDefinition, CostEstimate,
# ValidationSchema, ConsentInfo, ConsentScope,
# CostContext, LLMInstruction
router.py # Parent RouterMCP - security pipeline + dispatch
middleware.py # CircuitBreaker, ConsentGate, CostGate,
# AuditLog, TokenLedger, ParamValidator
storage.py # BaseStorage - thread-safe SQLite with WAL
children/ # Domain-specific children
running/
__init__.py # Exports RunningChild
child.py # RunningChild(ChildMCP) - 13 tools, 3 tiers
processing.py # RunningProcessing - all analytics (stateless)
strava_api.py # OAuth + rate-limited Strava API calls
```

3. Security Pipeline (5 Middleware Layers)

Every tool call passes through these layers in order (cheapest checks first). The key design principle: **behavioral rules live on the server, not in the LLM**. Any LLM client (Claude, GPT, Gemini, open-source) gets the same enforcement automatically.

Layer	Class	Purpose	Scope
1. Param Validation	ParamValidator	Type, range, pattern checks. Rejects bad input before work.	Per-call
2. Circuit Breaker	CircuitBreaker	Auto-blocks domain after 3 consecutive failures. Resets after 5 min.	Per-domain
3. Consent Gate	ConsentGate	Per-domain biometric consent. Session-scoped, revocable. Running != CGM.	Per-domain, per-session
4. Cost Gate	CostGate	Pre-estimates tokens before execution. Shows full vs. downsampled options.	Tier 3 only
5. Audit Log	AuditLog + TokenLedger	Every call logged to SQLite. Cumulative token spend by domain.	All calls

Consent Gate Design

The consent gate returns a structured JSON response containing: a **user_prompt** (the exact text to show the user), **llm_instruction** (machine-readable behavioral contract with `must_do`, `must_not_do`, and `on_ambiguous_reply` directives), **scope** (duration, revocability), and **dynamic scope** (distinguishes what THIS call needs vs what the SESSION will allow).

This design ensures the LLM cannot auto-approve, bundle other questions in the consent turn, or paraphrase the prompt in a way that hides scope. The instruction contract is individually checkable — "technically comply while spiritually violating" requires deliberate effort.

Cost Gate Design

The cost gate calls `child.estimate_cost()` BEFORE execution (no wasted computation). If tokens exceed threshold (default 35,000), it returns human-relatable context: "~50x a typical analysis call" and "~10x more than the downsampled alternative". Raw token counts are never presented alone — always anchored to something meaningful.

4. Three-Tier Access Model

Progressive disclosure: 90%+ of questions are answered at Tier 1 with zero biometric data leaving the machine. Raw streams are only exposed when explicitly needed AND after informed consent AND cost approval.

Tier	What the LLM Sees	Tokens	Gate	Coverage
Tier 1 - Free	Server-computed reports (splits, zones, drift, decoupling, EF, trends, anomalies)	200-1,500	None	~90% of questions
Tier 2 - Gated	Downsampled streams at 10-15s intervals for visualization	3,000-7,000	Biometric consent	Charts, overlays
Tier 3 - Cost	Per-second streams with precision reduction + selective filtering	25,000-60,000	Consent + cost approval	Fine-grained investigation

Complete Tool Inventory (Running Child)

Tool	Tier	Description	~Tokens
strava_sync	1	Pull recent activities from Strava into local cache	~50
strava_list_runs	1	List recent running activities with summary stats	~300
strava_activity_detail	1	Single activity overview (distance, time, HR, elevation)	~200
strava_hr_analysis	1	Zone distribution, drift, anomalies. Raw data stays on server.	~300
strava_pace_analysis	1	Mile splits, run/walk classification	~300
strava_stop_analysis	1	Pause detection with GPS locations + saved labels	~200
strava_label_stop	1	Save what happened at a stop (persists to SQLite)	~50
strava_run_report	1	Comprehensive: decoupling, EF, drift, phases, GAP splits, anomalies	~800
strava_trend_report	1	Rolling weekly volume, avg pace, avg HR, longest run	~600
strava_compare_runs	1	Side-by-side comparison of 2-5 runs: pace, HR, drift, EF	~1,500
strava_downsampled_streams	2	HR, pace, GPS at 10-15s intervals for visualization	3,000-7,000
strava_full_streams	3	Per-second data with precision reduction, selective streams	25,000-60,000

5. Framework Layer (Domain-Agnostic)

The framework handles all cross-cutting concerns. It knows nothing about running, Strava, or any specific biosensor. A new child registers with the router and the framework automatically provides consent gates, cost gates, audit logging, and circuit breaking.

interfaces.py — Core Abstractions

ChildMCP (ABC): Base class with abstract properties `domain`, `display_name`, `tool_definitions`, `param_schemas`, and abstract methods `execute()` and `estimate_cost()`. Also provides `consent_info` and `data_types_for_tool()` for dynamic scope.

ToolDefinition: `name`, `tier` (1/2/3), `description`, `params` dict. **CostEstimate**: `tokens`, `has_cheaper_alternative`, `alternative_tokens`, `alternative_description`. **ValidationSchema**: `type`, `required`, `default`, `min/max`, `pattern`, `allowed_values`.

LLMInstruction: Structured behavioral contract with `must_do` (list), `must_not_do` (list), `on_ambiguous_reply` (string). Serializes to ~150-250 tokens. **ConsentInfo**: `data_types`, `purpose`, `scope` (ConsentScope). **ConsentScope**: `duration`, `covers_future_calls`, `revocable`, `revoke_instruction`. **CostContext**: `tokens`, `relative_to_typical`, `relative_to_cheaper_pct`, `estimated_cost_usd`.

router.py — RouterMCP

The RouterMCP creates an MCP Server with unified tool listing and dispatch. It dynamically generates `approve_consent_[domain]` and `revoke_consent_[domain]` tools for each registered child. The dispatch pipeline runs layers 1-5 in order, short-circuiting on any gate trigger. Successful results have `_meta` attached with `tokens_this_call`, `session_total_tokens`, `domain`, and `tier`.

middleware.py — Security Components

CircuitBreaker: Per-key (domain) failure tracking. Trips after threshold consecutive failures, auto-resets after cooldown. **ConsentGate**: In-memory dict of approved domains. Session-scoped (resets on restart). **CostGate**: Threshold comparison with `humanize()` for relatable cost context. **AuditLog**: SQLite with `threading.local()` connections. **TokenLedger**: In-memory session spend tracker with `by_domain()` breakdown. **ParamValidator**: Static `validate()` method against `ValidationSchema` definitions. **estimate_tokens()**: ~4 chars per token heuristic for JSON payloads.

storage.py — BaseStorage

Thread-safe SQLite with WAL mode and per-thread connections via `threading.local()`. Children override `_schema_sql()` to define domain-specific tables. Provides `execute()`, `executemany()`, `commit()`, `fetchone()`, `fetchall()` convenience methods. Eagerly creates schema on init. Cache size set to 8MB.

6. Running Child Implementation

RunningStorage (extends BaseStorage)

Tables: **activities** (id, data JSON, synced_at), **streams** (activity_id, data JSON, fetched_at with TTL eviction), **stop_labels** (activity_id + stop_number PK, label, notes, labeled_at). Stream cache TTL configurable via STRAVA_STREAM_CACHE_TTL_DAYS env var (default 7).

RunningChild (extends ChildMCP)

Domain: "running". Display name: "Running (Strava)". Loads user config from ~/.strava-coach/user_config.json for max_hr (default 195) and resting_hr (default 60). ConsentInfo declares data_types: [heart rate, GPS location, pace, elevation] with purpose "training analysis and visualization". Implements data_types_for_tool() for dynamic scope narrowing on stream requests.

Stream-Type to Data-Type Mapping (Dynamic Consent Scope)

Stream	Data Types
heartrate	heart rate
velocity_smooth	pace
latlng	GPS location
altitude, grade_smooth	elevation
distance	pace
time, moving	(none - metadata only)

When a user requests only heartrate streams, the consent prompt says "This call needs: heart rate" instead of listing all 4 data types. Session-wide consent still covers all types.

7. Server-Side Analytics Engine

All methods in RunningProcessing are stateless pure functions. Raw per-second data stays on the server; only computed metrics reach the LLM.

Precision Reduction

Stream	Precision	Rationale
GPS (latlng)	5 decimals	1.1m accuracy - tighter than GPS hardware
velocity_smooth	2 decimals	Loses 0.005 m/s - undetectable
altitude	Integer	Sub-meter irrelevant for running
distance	Integer	Sub-meter irrelevant for splits
grade_smooth	1 decimal	PRESERVED - protects GAP calculations ($\text{cost} = 1 + 0.03 * \text{grade\%}$)
heartrate, time, moving	Unchanged	Already low-precision

Downsampling (Tier 2)

Samples every N seconds (default 10s). For a 15mi run: 8,600 points -> ~860 points. Always includes first and last point. Preserves curve shape for visualization. Combined with precision reduction for maximum token savings.

Computed Metrics

Metric	Method	What It Computes
HR Zones	compute_hr_zones()	5-zone distribution (60/70/80/90/100% of configurable max_hr), avg/max/min HR
HR Drift	compute_hr_drift()	First-half vs second-half avg HR. <5% aerobic, 5-10% moderate, >10% significant
Decoupling	compute_decoupling()	HR:pace ratio first half vs second half. <5% well coupled, >10% decoupled
Efficiency Factor	compute_efficiency_factor()	EF = normalized pace / avg HR * 1000. Higher = more efficient
Mile Splits	compute_mile_splits()	Per-mile pace from distance array. Includes partial final mile if >100m
GAP Splits	compute_gap_splits()	Grade Adjusted Pace: $\text{cost} = 1 + 0.03 * \text{grade\%}$. Uses mile splits internally

Run Phases	detect_run_phases()	30s rolling avg velocity. Classifies: easy (<80% avg), steady, tempo, fast (>115% avg)
Anomalies	detect_anomalies()	HR flatline (same value 60+ sec) + spike (>30 bpm in 5 sec, 30s cooldown)
Stop Detection	detect_stops()	Velocity <0.5 m/s for 10+ sec. Classifications: brief/short/extended. Home proximity.
Token Estimation	estimate_stream_tokens())	Per-stream char estimates * 1.2 overhead / 4. No serialization needed.

8. Strava API & Token Management

StravaAPI handles OAuth token management, auto-refresh, and rate-limit awareness. Strava enforces 100 requests per 15-minute window and 1,000 per day. The client tracks timestamps and raises BEFORE hitting the limit (proactive, not reactive).

Rate limit timestamps are persisted to `~/.strava-coach/rate_limit.json` so the counter survives process restarts. Token file permissions are secured with `icacfs` (Windows) or `chmod 600` (Unix). Request timeout is set to (5s connect, 30s read).

OAuth Setup Wizard

wizard.py runs a temporary localhost HTTP server on port 8189 to capture the OAuth callback. Opens the user's browser to Strava's authorization page. Exchanges the auth code for tokens. Saves tokens to `~/.strava-coach/tokens.json` with platform-appropriate file permissions. Scopes: read, activity:read_all.

9. CLI & Installation

CLI Commands (___main___.py)

Command	Function	What It Does
strava-coach serve	cmd_serve()	Start MCP server via stdio. Creates RouterMCP, registers RunningChild, runs.
strava-coach setup	cmd_setup()	Run OAuth setup wizard (browser-based).
strava-coach status	cmd_status()	Diagnostic: config dir, tokens, user config, database, audit log, Claude config.
strava-coach uninstall	cmd_uninstall()	Remove config dir + Claude Desktop MCP registration.

Configuration

CONFIG_DIR: ~/.strava-coach (override via STRAVA_CONFIG_DIR env var). DATA_DIR: CONFIG_DIR/data (override via STRAVA_DATA_DIR env var). LOG_DIR: CONFIG_DIR/logs. Logging uses RotatingFileHandler (5MB, 3 backups) + stderr StreamHandler (stdout is the MCP wire).

Claude Desktop Integration

```
// claude_desktop_config.json
{
  "mcpServers": {
    "strava-coaching": {
      "command": "~/strava-coach/venv/bin/python",
      "args": ["-m", "strava_coach", "serve"],
      "env": {
        "STRAVA_CONFIG_DIR": "~/strava-coach",
        "STRAVA_DATA_DIR": "~/strava-coach/data"
      }
    }
  }
}
```

10. Key Design Decisions & Rationale

Behavioral rules on the server, not in the LLM

Protocol enforcement (access tiers, consent gates, cost gates) lives in tool descriptions server-side so any AI client gets the same enforcement automatically. Not dependent on prompt engineering or any specific LLM's instruction-following quality.

Cheapest checks first in the pipeline

Param validation (near-zero cost) runs before circuit breaker, which runs before consent gate, which runs before cost estimation. Failed requests exit as early and cheaply as possible.

Pre-estimation, not post-billing

Cost gate calls `estimate_cost()` before execution. Uses stream metadata (point count * stream count) rather than serializing the full payload. No wasted computation on rejected requests.

Per-domain consent isolation

Consenting to running data does NOT auto-unlock CGM or sleep data. Each biosensor domain requires independent authorization. Prevents scope creep across data types.

Session-scoped, revocable consent

Consent does not persist across sessions. Users can revoke mid-session. Matches IRB expectations for informed consent. Consent state is in-memory only.

Structured LLMInstruction over freeform strings

Instead of a text paragraph telling the LLM what to do, the gate returns a JSON object with individually checkable `must_do`, `must_not_do`, and `on_ambiguous_reply` fields. Makes compliance auditable and drift detectable.

Grade precision preserved at 1 decimal

GAP calculation uses $\text{cost} = 1 + 0.03 * \text{grade\%}$. Rounding grade to integer introduces ~3% error in split calculations. All other numeric types are reduced more aggressively.

0.5 m/s stop detection threshold

0.3 m/s was too aggressive (flagged slow shuffles at end of hard efforts). 0.5 m/s (~1.8 km/h) reliably detects "completely stopped" without false positives.

Apple Watch HR artifact awareness

Watch can lock HR at a fixed value (e.g., 160 bpm for ~7 minutes) during wrist-lock conditions. Subsequent spikes/drops are sensor catchup, not real cardiac events. Flatline detection (60s+ same value) and spike detection (30 bpm / 5s with 30s cooldown) are designed around this known artifact.

Spike detection 30-second cooldown

Without cooldown, a single sensor catchup burst can generate dozens of overlapping anomaly entries. The cooldown ensures one entry per actual event.

orjson with stdlib fallback

orjson is ~3x faster for JSON serialization. Falls back to stdlib json if not installed. Transparent to all consumers via `_dumps/_loads` wrappers.

GI absorption degrades at high HR

Fueling stops above HR 170 yield only ~15-25% absorption (graded WASTED). Optimal fueling window is HR under 150 (90-100% absorption). This informs stop analysis interpretation.

11. Known Issues & Diagnostic Findings

A comprehensive diagnostic was performed on 2026-04-09. Key findings (many have been addressed in the v3.0.0 codebase):

#	Severity	Issue	Status
1	Fixed	Wrong Claude Desktop config path for Windows Store	PS1 installer now detects Store paths
2	Open	GitHub username placeholders in remote install scripts	Local ZIP install works; GitHub needs username
3	Fixed	setup wizard fails after pip install (path walk)	Wizard now bundled as strava_coach.wizard module
4	Fixed	os.chmod does nothing on Windows	Now uses icacls on Windows
5	Fixed	No timeout on Strava API requests	Timeout set to (5s, 30s)
6	Fixed	Rate limit counter resets on restart	Timestamps persisted to rate_limit.json
7	Fixed	AuditLog single shared connection	Now uses threading.local()
8	Fixed	home_coords never passed to stop detection	Now reads from user_config.json
9	Fixed	Stop threshold 0.3 m/s vs designed 0.5 m/s	Updated to 0.5 m/s
10	Fixed	Stop classifications not implemented	brief/short/extended now in output
11	Fixed	Spike detection too aggressive	30-second cooldown added
12	Fixed	ConsentInfo not exported from framework	Added to __init__.py exports
14	Fixed	bc dependency in install.sh version check	Uses Python for version check
15	Fixed	PowerShell execution policy not checked	Policy check added to PS1
16	Fixed	get_user_config in wrong layer	Moved to RunningChild
17	Fixed	StreamHandler not explicit about stderr	Explicit sys.stderr added

12. Extensibility: Adding New Biosensor Children

Adding a new biosensor domain requires implementing 4 abstract methods/properties and registering with the router. The framework handles all security, consent, cost gating, and audit automatically.

```
from strava_coach.framework import ChildMCP, ToolDefinition, CostEstimate,
ValidationSchema, ConsentInfo, ConsentScope

class CGMChild(ChildMCP):
    @property
    def domain(self) -> str: return "cgm"

    @property
    def display_name(self) -> str: return "Glucose (Dexcom)"

    @property
    def consent_info(self) -> ConsentInfo:
        return ConsentInfo(
            data_types=["glucose levels", "meal markers"],
            purpose="glycemic analysis and trends",
        )

    @property
    def tool_definitions(self) -> list[ToolDefinition]:
        return [
            ToolDefinition("cgm_daily_report", 1,
                "Time-in-range, variability, avg glucose", {...}),
            ToolDefinition("cgm_raw_readings", 3,
                "Per-minute glucose values", {...}),
        ]

    async def execute(self, tool_name, params) -> dict: ...
    async def estimate_cost(self, tool_name, params) -> CostEstimate: ...

# Register in __main__.py:
router.register_child(CGMChild(config_dir, data_dir))
# Router auto-generates approve_consent_cgm + revoke_consent_cgm
```

Potential Future Children

Domain	Data Source	Example Tier 1 Metrics	Example Tier 3 Data
CGM	Dexcom/Libre	Time-in-range, glycemic variability, meal response	Per-minute glucose readings
Sleep	Oura/Whoop/Apple Watch	Stage duration, efficiency, latency, fragmentation	Per-epoch staging data
ECG	Apple Watch/Kardia	Rhythm classification, HRV (RMSSD, SDNN), QT intervals	Raw ECG waveform
Lab	Blood panels	Out-of-range flags, longitudinal trends	Full panel values

13. Full Source File Index

File	Lines	Purpose
src/strava_coach/__init__.py	~12	Package metadata, version 3.0.0
src/strava_coach/__main__.py	~240	CLI: serve, setup, status, uninstall commands
src/strava_coach/wizard.py	~178	OAuth setup wizard with localhost callback server
src/strava_coach/framework/__init__.py	~32	Public API exports for framework layer
src/strava_coach/framework/interfaces.py	~256	ChildMCP ABC, ToolDefinition, CostEstimate, ValidationSchema, ConsentInfo, ConsentScope, CostContext, LLMInstruction
src/strava_coach/framework/router.py	~522	RouterMCP: MCP server creation, security pipeline dispatch, consent/revocation handlers
src/strava_coach/framework/middleware.py	~365	CircuitBreaker, ConsentGate, CostGate, AuditLog, TokenLedger, ParamValidator, estimate_tokens
src/strava_coach/framework/storage.py	~78	BaseStorage: thread-safe SQLite with WAL, connection pooling
src/strava_coach/children/__init__.py	~14	Children package docstring
src/strava_coach/children/running/__init__.py	~12	Exports RunningChild
src/strava_coach/children/running/child.py	~800	RunningChild: 13 tools, RunningStorage, all handlers
src/strava_coach/children/running/processing.py	~553	RunningProcessing: all analytics (stateless pure functions)
src/strava_coach/children/running/strava_api.py	~160	StravaAPI: OAuth, rate limiting, API calls
pyproject.toml	~30	Build config: deps (mcp, requests, orjson), entry point
install.sh	~100	Mac/Linux installer (local or GitHub)
install.ps1	~120	Windows installer with Store path detection

End of project context document. Upload this PDF to any Claude conversation to provide full architectural context for the biosensor-to-LLM optimization project.